4 MAGGIO 2026

REPORT DI VALIDAZIONE AVALLA E ATGT DEL MODELLO ASM DELL'ASCENSORE

MAFFEIS RICCARDO 1085706

Tema scelto	Ascensore a più piani con un'unica cabina
Obiettivo	Validazione del modello ASM
Metodo	Scenari AVALLA manuali e scenari generati con ATGT
Scenari considerati	10 scenari manuali e 5 scenari generati automaticamente
Esito	Tutti gli scenari validati terminano senza errori

Sommario

Validazione tramite scenari AVALLA 2
Scenari considerati 2
Esito complessivo 2
Dettaglio degli scenari validati 3
1. Idle ascensore 3
2. Richiesta verso piano superiore 5
3. Richiesta verso piano inferiore 8
4. Overload 11
5. Risoluzione overload 14
6. Richieste durante overload 17
7. Guasto tecnico 19
8. Blocco delle richieste durante guasto 21
9. Conservazione delle richieste durante guasto 23
10. Ripristino dopo guasto 25
Generazione automatica di scenari con ATGT 28
Conclusione 31

Validazione tramite scenari AVALLA

Questo documento presenta i risultati della validazione del modello ASM relativo al sistema di ascensore.

La validazione è stata eseguita mediante scenari AVALLA, ciascuno dedicato a una specifica situazione operativa del sistema. Gli scenari sono stati validati con coverage, in modo da osservare sia l'esito dei controlli sia la copertura delle regole del modello.

Scenari considerati

La validazione è stata suddivisa in più scenari, così da verificare separatamente le principali funzionalità del modello. Gli scenari considerati sono i seguenti:

- funzionamento in assenza di richieste;
- richiesta verso un piano superiore;
- richiesta verso un piano inferiore;
- gestione del sovraccarico;
- risoluzione del sovraccarico;
- acquisizione di richieste durante il sovraccarico;
- ingresso nello stato di guasto;
- blocco delle nuove richieste durante il guasto;
- conservazione delle richieste già acquisite durante il guasto;
- ripristino dopo guasto.

Ogni scenario è contenuto in un file .avalla separato, in modo che la simulazione parta sempre dallo stato iniziale definito nel modello ASM.

Esito complessivo

Tutti gli scenari validati terminano senza errori.

Nel complesso, la validazione conferma che il modello rispetta i principali requisiti attesi:

- la cabina non si muove con porte aperte;
- le richieste vengono acquisite e mantenute finché non sono servite;
- il movimento avviene in modo coerente con la direzione scelta;
- lo stato di OVERLOAD blocca la cabina e mantiene le porte aperte;
- lo stato di GUASTO blocca la cabina e sospende l'acquisizione di nuove richieste;
- le richieste già acquisite non vengono eliminate durante le anomalie;
- il sistema può tornare operativo dopo il timeout di ripristino.

Dettaglio degli scenari validati

1. Idle ascensore

File AVALLA: scenario_idle.avalla

Obiettivo

Verificare che, in assenza di richieste e di condizioni di errore, l'ascensore rimanga fermo nello stato di attesa.

Lo scenario controlla che la cabina non cambi piano, che le porte restino chiuse e che la direzione rimanga NESSUNA.

Simulazione eseguita

Lo scenario non genera nuove richieste e non attiva eventi di guasto.

Il sistema deve quindi mantenere lo stato di inattività.

```
eventoGuasto = false
```

Stati osservati dalla simulazione

Dopo il passo logico eseguito, lo stato significativo osservato è:

```
pianoCorrente = 0
statoCabina = FERMA
statoPorte = CHIUSE
direzione = NESSUNA
statoErrore = NESSUNO
```

La console mostra inoltre che viene eseguita la regola di idle:

```
statoCabina = FERMA
direzione = NESSUNA
```

Check eseguiti

Check	Esito
pianoCorrente = 0	Passed
statoCabina = FERMA	Passed
statoPorte = CHIUSE	Passed
direzione = NESSUNA	Passed
statoErrore = NESSUNO	Passed

Coverage dello scenario

Regola	Branch coverage	Rule coverage	Update rule coverage	Forall rule coverage
r_gestisciAscensore	50.00%	50.00%	-	-
r_aggiornaPersone	25.00%	25.00%	0.00%	-
r_idle	-	100.00%	0.00%	-
r_gestisciGuasto	33.33%	12.50%	0.00%	-
r_main	50.00%	100.00%	-	-
r_acquisisciRichieste	50.00%	50.00%	0.00%	33.33%
r_gestisciErrori	50.00%	66.67%	-	-

Regole non coperte

Le seguenti regole non vengono coperte da questo scenario:

`r_gestisciOverload`

`r_chiudiPorte`

`r_scegliDirezione`

`r_muoviAscensore`

`r_serviPianoCorrente`

Questo risultato è coerente con l'obiettivo dello scenario, perché il test verifica esclusivamente il comportamento del sistema in assenza di richieste.

Le regole relative al movimento, alla scelta della direzione, al servizio dei piani, alla chiusura delle porte e al sovraccarico vengono invece coperte da altri scenari.

Osservazione

Lo scenario conferma che, quando non sono presenti richieste attive e non ci sono errori, il modello mantiene l'ascensore nello stato di idle.

La cabina resta ferma al piano iniziale, le porte rimangono chiuse e la direzione resta NESSUNA.

2. Richiesta verso piano superiore

File AVALLA: scenario_richiesta_piano_superiore.avalla

Obiettivo

Verificare che l'ascensore sia in grado di acquisire e servire una richiesta interna verso un piano superiore.

Lo scenario controlla che la richiesta per il piano 3 venga registrata, che la direzione venga impostata a SU, che la cabina si muova progressivamente verso l'alto e che, una volta raggiunto il piano richiesto, la richiesta venga rimossa.

Simulazione eseguita

Lo scenario parte dallo stato iniziale del modello:

```
pianoCorrente = 0
statoCabina = FERMA
statoPorte = CHIUSE
direzione = NESSUNA
statoErrore = NESSUNO
richiesteAttive(3) = false
```

Successivamente viene generata una richiesta interna verso il piano 3.

```
pulsanteInterno(3) = true
```

Il sistema deve acquisire la richiesta, impostare la direzione verso l'alto e muovere la cabina fino al piano richiesto.

Stati osservati dalla simulazione

Dopo l'acquisizione della richiesta, la cabina inizia a muoversi verso l'alto:

```
richiesteAttive(3) = true
direzione = SU
pianoCorrente = 1
statoCabina = IN_MOVIMENTO
statoPorte = CHIUSE
```

Nel passo successivo la cabina prosegue verso il piano 3:

```
pianoCorrente = 2
direzione = SU
statoPorte = CHIUSE
```

Successivamente raggiunge il piano richiesto:

```
pianoCorrente = 3
direzione = SU
statoPorte = CHIUSE
```

Al passo logico seguente, il piano viene servito:

```
pianoCorrente = 3
richiesteAttive(3) = false
statoCabina = FERMA
statoPorte = APERTE
direzione = NESSUNA
```

Infine, le porte vengono chiuse:

```
pianoCorrente = 3
statoCabina = FERMA
statoPorte = CHIUSE
direzione = NESSUNA
```

Check eseguiti

Check	Esito
pianoCorrente = 0	Passed
statoCabina = FERMA	Passed
statoPorte = CHIUSE	Passed
direzione = NESSUNA	Passed
statoErrore = NESSUNO	Passed
richiesteAttive(3) = false	Passed
richiesteAttive(3) = true	Passed
direzione = SU	Passed
pianoCorrente = 1	Passed
statoCabina = IN_MOVIMENTO	Passed
statoPorte = CHIUSE	Passed
pianoCorrente = 2	Passed
direzione = SU	Passed
pianoCorrente = 3	Passed
direzione = SU	Passed
richiesteAttive(3) = false	Passed
statoCabina = FERMA	Passed
statoPorte = APERTE	Passed
direzione = NESSUNA	Passed

Check	Esito
statoPorte = CHIUSE dopo la chiusura	Passed

Coverage dello scenario

Regola	Branch coverage	Rule coverage	Update rule coverage	Forall rule coverage
r_gestisciAscensore	75.00%	90.00%	-	-
r_aggiornaPersone	75.00%	75.00%	0.00%	-
r_chiudiPorte	50.00%	100.00%	100.00%	-
r_gestisciGuasto	33.33%	12.50%	0.00%	-
r_scegliDirezione	33.33%	36.84%	10.00%	-
r_main	50.00%	100.00%	-	-
r_acquisisciRichieste	100.00%	100.00%	100.00%	66.67%
r_muoviAscensore	33.33%	41.67%	33.33%	-
r_gestisciErrori	50.00%	66.67%	-	-
r_serviPianoCorrente	50.00%	100.00%	100.00%	-

Regole non coperte

Le seguenti regole non vengono coperte da questo scenario:

`r_gestisciOverload`

`r_idle`

Questo risultato è coerente con l'obiettivo dello scenario, perché il test riguarda il movimento e il servizio di una richiesta verso un piano superiore.

Le regole relative al sovraccarico e allo stato di inattività vengono invece coperte da altri scenari.

Osservazione

Lo scenario conferma che il modello gestisce correttamente una richiesta interna verso un piano superiore.

La richiesta del piano 3 viene acquisita, la direzione viene impostata a SU e la cabina si muove progressivamente dal piano 0 al piano 3. Una volta raggiunto il piano richiesto, la cabina si ferma, apre le porte e rimuove la richiesta dalle richieste attive.

3. Richiesta verso piano inferiore

File AVALLA: scenario_richiesta_piano_inferiore.avalla

Obiettivo

Verificare che l'ascensore sia in grado di servire una richiesta posta sotto il piano corrente.

Lo scenario porta inizialmente la cabina al piano 3 e successivamente genera una chiamata esterna verso il basso dal piano 0.

Simulazione eseguita

La simulazione viene divisa in due fasi.

Nella prima fase viene generata una richiesta interna verso il piano 3, così da portare la cabina a un piano superiore.

```
pulsanteInterno(3) = true
```

Dopo aver raggiunto il piano 3, viene generata una chiamata esterna verso il basso dal piano 0.

```
chiamataPianoGiu(0) = true
```

Il sistema deve acquisire la richiesta, impostare la direzione a GIU e muovere progressivamente la cabina fino al piano 0.

Stati osservati dalla simulazione

Durante la prima fase, la cabina si muove verso il piano 3:

```
pianoCorrente = 1
direzione = SU
statoCabina = IN_MOVIMENTO
richiesteAttive(3) = true
```

Successivamente raggiunge il piano 3, serve la richiesta e apre le porte:

```
pianoCorrente = 3
statoCabina = FERMA
statoPorte = APERTE
richiesteAttive(3) = false
direzione = NESSUNA
```

Dopo la chiusura delle porte, viene acquisita la chiamata dal piano 0:

```
richiesteAttive(0) = true
direzione = GIU
pianoCorrente = 2
statoCabina = IN_MOVIMENTO
statoPorte = CHIUSE
```

La cabina prosegue verso il basso:

```
pianoCorrente = 1
direzione = GIU
```

poi raggiunge il piano 0:

```
pianoCorrente = 0
direzione = GIU
```

Infine il piano richiesto viene servito:

```
pianoCorrente = 0
richiesteAttive(0) = false
statoCabina = FERMA
statoPorte = APERTE
direzione = NESSUNA
```

Check eseguiti

Check	Esito
pianoCorrente = 3	Passed
statoPorte = CHIUSE	Passed
richiesteAttive(0) = true	Passed
direzione = GIU	Passed
pianoCorrente = 2	Passed
statoCabina = IN_MOVIMENTO	Passed
pianoCorrente = 1	Passed
direzione = GIU	Passed
pianoCorrente = 0	Passed
direzione = GIU	Passed
richiesteAttive(0) = false	Passed
statoCabina = FERMA	Passed
statoPorte = APERTE	Passed
direzione = NESSUNA	Passed

Coverage dello scenario

Regola	Branch coverage	Rule coverage	Update rule coverage	Forall rule coverage
r_gestisciAscensore	75.00%	90.00%	-	-
r_aggiornaPersone	75.00%	75.00%	0.00%	-
r_chiudiPorte	50.00%	100.00%	100.00%	-
r_gestisciGuasto	33.33%	12.50%	0.00%	-
r_scegliDirezione	55.56%	57.89%	20.00%	-
r_main	50.00%	100.00%	-	-
r_acquisisciRichieste	100.00%	100.00%	100.00%	66.67%
r_muoviAscensore	66.67%	75.00%	66.67%	-
r_gestisciErrori	50.00%	66.67%	-	-
r_serviPianoCorrente	50.00%	100.00%	100.00%	-

Regole non coperte

Le seguenti regole non vengono coperte da questo scenario:

`r_gestisciOverload`

`r_idle`

Questo risultato è coerente con l'obiettivo dello scenario, perché il test riguarda il movimento e il servizio di una richiesta verso un piano inferiore.

Le regole relative al sovraccarico e allo stato di inattività vengono invece coperte da altri scenari.

Osservazione

Lo scenario conferma che il modello gestisce correttamente una richiesta posta sotto il piano corrente.

La cabina acquisisce la chiamata dal piano 0, imposta la direzione a GIU, si muove progressivamente verso il basso e, una volta raggiunto il piano richiesto, si ferma, apre le porte e rimuove la richiesta dalle richieste attive.

4. Overload

File AVALLA: scenario_overload.avalla

Obiettivo

Verificare che il sistema entri correttamente nello stato OVERLOAD quando il numero di persone presenti in cabina supera la capacità massima consentita.

Lo scenario controlla che, in caso di sovraccarico, la cabina venga bloccata, le porte restino aperte e la direzione venga annullata.

Simulazione eseguita

Lo scenario apre inizialmente le porte al piano corrente e successivamente simula l'ingresso di un numero di persone superiore alla capacità massima.

```
personeEntrate = 9
personeUscite = 0
```

Poiché la capacità massima del modello è pari a 8, il sistema deve entrare nello stato OVERLOAD.

Stati osservati dalla simulazione

Prima del sovraccarico, lo stato significativo osservato è:

```
pianoCorrente = 0
statoCabina = FERMA
statoPorte = APERTE
richiesteAttive(0) = false
```

Dopo l'ingresso delle persone, il sistema rileva il sovraccarico:

```
numeroPersone = 9
statoErrore = OVERLOAD
statoCabina = BLOCCATA
statoPorte = APERTE
direzione = NESSUNA
```

Check eseguiti

Check	Esito
pianoCorrente = 0	Passed
statoPorte = APERTE	Passed
statoCabina = FERMA	Passed
richiesteAttive(0) = false	Passed
numeroPersone = 9	Passed
statoErrore = OVERLOAD	Passed
statoCabina = BLOCCATA	Passed
statoPorte = APERTE	Passed
direzione = NESSUNA	Passed

Coverage dello scenario

Regola	Branch coverage	Rule coverage	Update rule coverage	Forall rule coverage
r_gestisciOverload	25.00%	50.00%	25.00%	-
r_gestisciAscensore	37.50%	40.00%	-	-
r_aggiornaPersone	75.00%	75.00%	50.00%	-
r_gestisciGuasto	33.33%	12.50%	0.00%	-
r_main	75.00%	100.00%	-	-
r_acquisisciRichieste	100.00%	100.00%	100.00%	66.67%
r_gestisciErrori	100.00%	100.00%	-	-
r_serviPianoCorrente	50.00%	100.00%	50.00%	-

Regole non coperte

Le seguenti regole non vengono coperte da questo scenario:

`r_chiudiPorte`

`r_idle`

`r_scegliDirezione`

`r_muoviAscensore`

Questo risultato è coerente con l'obiettivo dello scenario, perché il test è focalizzato sulla gestione del sovraccarico.

Le regole relative alla chiusura delle porte, allo stato di inattività, alla scelta della direzione e al movimento della cabina vengono invece coperte da altri scenari.

Osservazione

Lo scenario conferma che il modello rileva correttamente la condizione di sovraccarico.

Quando il numero di persone presenti in cabina supera la capacità massima, il sistema entra nello stato OVERLOAD, blocca la cabina, mantiene le porte aperte e imposta la direzione a NESSUNA.

5. Risoluzione overload

File AVALLA: scenario_risoluzione_overload.avalla

Obiettivo

Verificare che il sistema esca correttamente dallo stato OVERLOAD quando il numero di persone presenti in cabina torna entro la capacità massima consentita.

Lo scenario controlla che, dopo la discesa di una persona, il numero di persone passi da 9 a 8 e che il sistema torni allo stato operativo.

Simulazione eseguita

Lo scenario genera inizialmente una condizione di sovraccarico simulando l'ingresso di un numero di persone superiore alla capacità massima.

```
personeEntrate = 9
personeUscite = 0
```

Il sistema entra quindi nello stato OVERLOAD.

Successivamente viene simulata l'uscita di una persona dalla cabina:

```
personeEntrate = 0
personeUscite = 1
```

Poiché la capacità massima è pari a 8, il sistema deve uscire dallo stato di sovraccarico e tornare operativo.

Stati osservati dalla simulazione

Dopo l'ingresso delle persone, il sistema rileva il sovraccarico:

```
numeroPersone = 9
statoErrore = OVERLOAD
statoCabina = BLOCCATA
statoPorte = APERTE
direzione = NESSUNA
```

Dopo l'uscita di una persona, il numero di persone rientra nella capacità massima:

```
numeroPersone = 8
statoErrore = NESSUNO
statoCabina = FERMA
statoPorte = CHIUSE
direzione = NESSUNA
```

Check eseguiti

Check	Esito
numeroPersone = 9	Passed
statoErrore = OVERLOAD	Passed
statoCabina = BLOCCATA	Passed
statoPorte = APERTE	Passed
numeroPersone = 8	Passed
statoErrore = NESSUNO	Passed
statoCabina = FERMA	Passed
statoPorte = CHIUSE	Passed
direzione = NESSUNA	Passed

Coverage dello scenario

Regola	Branch coverage	Rule coverage	Update rule coverage	Forall rule coverage
r_gestisciOverload	75.00%	100.00%	62.50%	-
r_gestisciAscensore	62.50%	60.00%	-	-
r_aggiornaPersone	75.00%	75.00%	50.00%	-
r_idle	-	100.00%	0.00%	-
r_gestisciGuasto	33.33%	12.50%	0.00%	-
r_main	75.00%	100.00%	-	-
r_acquisisciRichieste	100.00%	100.00%	100.00%	66.67%
r_gestisciErrori	100.00%	100.00%	-	-
r_serviPianoCorrente	50.00%	100.00%	50.00%	-

Regole non coperte

Le seguenti regole non vengono coperte da questo scenario:

`r_chiudiPorte`

`r_scegliDirezione`

`r_muoviAscensore`

Questo risultato è coerente con l'obiettivo dello scenario, perché il test è focalizzato sulla risoluzione della condizione di sovraccarico.

Le regole relative alla chiusura ordinaria delle porte, alla scelta della direzione e al movimento della cabina vengono invece coperte da altri scenari.

Osservazione

Lo scenario conferma che il modello gestisce correttamente l'uscita dallo stato OVERLOAD.

Quando il numero di persone passa da 9 a 8, rientrando nella capacità massima consentita, il sistema elimina lo stato di errore, riporta la cabina nello stato FERMA, chiude le porte e mantiene la direzione a NESSUNA.

6. Richieste durante overload

File AVALLA: scenario_richieste_durante_overload.avalla

Obiettivo

Verificare che, durante lo stato OVERLOAD, il sistema continui ad acquisire nuove richieste.

Lo scenario controlla che una richiesta interna verso il piano 2, effettuata mentre la cabina è bloccata per sovraccarico, venga comunque registrata tra le richieste attive.

Simulazione eseguita

Lo scenario genera inizialmente una condizione di sovraccarico simulando l'ingresso di un numero di persone superiore alla capacità massima.

```
personeEntrate = 9
personeUscite = 0
```

Successivamente, mentre il sistema si trova nello stato OVERLOAD, viene premuto il pulsante interno del piano 2.

```
pulsanteInterno(2) = true
```

Il sistema deve rimanere in stato di sovraccarico, ma deve comunque memorizzare la nuova richiesta.

Stati osservati dalla simulazione

Prima dell'ingresso nello stato di sovraccarico, lo stato significativo osservato è:

```
statoCabina = FERMA
statoPorte = APERTE
direzione = NESSUNA
richiesteAttive(0) = false
```

Dopo il superamento della capacità massima, il sistema entra nello stato OVERLOAD:

```
numeroPersone = 9
statoErrore = OVERLOAD
statoCabina = BLOCCATA
statoPorte = APERTE
direzione = NESSUNA
```

Durante lo stato OVERLOAD, viene acquisita la richiesta del piano 2:

```
statoErrore = OVERLOAD
statoCabina = BLOCCATA
statoPorte = APERTE
richiesteAttive(2) = true
```

Check eseguiti

Check	Esito
statoErrore = OVERLOAD	Passed
statoPorte = APERTE	Passed
statoCabina = BLOCCATA	Passed
richiesteAttive(2) = true	Passed

Coverage dello scenario

Regola	Branch coverage	Rule coverage	Update rule coverage	Forall rule coverage
r_gestisciOverload	25.00%	50.00%	25.00%	-
r_gestisciAscensore	37.50%	40.00%	-	-
r_aggiornaPersone	75.00%	75.00%	50.00%	-
r_gestisciGuasto	33.33%	12.50%	0.00%	-
r_main	75.00%	100.00%	-	-
r_acquisisciRichieste	100.00%	100.00%	100.00%	66.67%
r_gestisciErrori	100.00%	100.00%	-	-
r_serviPianoCorrente	50.00%	100.00%	50.00%	-

Regole non coperte

Le seguenti regole non vengono coperte da questo scenario:

`r_chiudiPorte`

`r_idle`

`r_scegliDirezione`

`r_muoviAscensore`

Questo risultato è coerente con l'obiettivo dello scenario, perché il test è focalizzato sulla gestione delle richieste durante il sovraccarico.

Le regole relative alla chiusura delle porte, allo stato di inattività, alla scelta della direzione e al movimento della cabina vengono invece coperte da altri scenari.

Osservazione

Lo scenario conferma che il modello distingue correttamente lo stato `OVERLOAD` dallo stato `GUASTO`.

Durante il sovraccarico, la cabina rimane bloccata e le porte restano aperte, ma il controllore continua ad acquisire nuove richieste. La richiesta del piano 2 viene quindi registrata correttamente tra le richieste attive.

7. Guasto tecnico

File AVALLA: scenario_guasto.avalla

Obiettivo

Verificare che il sistema entri correttamente nello stato GUASTO quando viene generato un evento di guasto.

Lo scenario controlla che, a seguito dell'attivazione del guasto, la cabina venga bloccata, le porte vengano chiuse, la direzione venga annullata e il timer di ripristino venga inizializzato

Simulazione eseguita

Lo scenario genera un evento di guasto sul sistema.

```
eventoGuasto = true
```

Il sistema deve entrare nello stato GUASTO e impedire il normale movimento della cabina.

Stati osservati dalla simulazione

Dopo l'attivazione del guasto, lo stato significativo osservato è:

```
statoErrore = GUASTO
statoCabina = BLOCCATA
statoPorte = CHIUSE
direzione = NESSUNA
timer = 10
```

Dopo un ulteriore aggiornamento della simulazione, il sistema rimane nello stato di guasto:

```
statoErrore = GUASTO
statoCabina = BLOCCATA
statoPorte = CHIUSE
direzione = NESSUNA
timer = 10
```

Check eseguiti

Check	Esito
statoErrore = GUASTO	Passed
statoCabina = BLOCCATA	Passed
statoPorte = CHIUSE	Passed
direzione = NESSUNA	Passed
timer = 10	Passed

Coverage dello scenario

Regola	Branch coverage	Rule coverage	Update rule coverage	Forall rule coverage
r_aggiornaPersone	25.00%	25.00%	0.00%	-
r_gestisciGuasto	33.33%	50.00%	27.27%	-
r_main	50.00%	85.71%	-	-
r_acquisisciRichieste	50.00%	50.00%	0.00%	33.33%
r_gestisciErrori	50.00%	66.67%	-	-

Regole non coperte

Le seguenti regole non vengono coperte da questo scenario:

`r_gestisciOverload`

`r_gestisciAscensore`

`r_chiudiPorte`

`r_idle`

`r_scegliDirezione`

`r_muoviAscensore`

`r_serviPianoCorrente`

Questo risultato è coerente con l'obiettivo dello scenario, perché il test verifica esclusivamente l'ingresso nello stato GUASTO.

Le regole relative al movimento, al servizio dei piani, allo stato di inattività e al sovraccarico vengono invece coperte da altri scenari specifici.

Osservazione

Lo scenario conferma che il modello gestisce correttamente l'attivazione di un guasto tecnico.

Quando viene generato l'evento di guasto, il sistema entra nello stato GUASTO, blocca la cabina, chiude le porte, annulla la direzione e inizializza il timer di ripristino a 10.

8. Blocco delle richieste durante guasto

File AVALLA: scenario_blocco_richieste_durante_guasto.avalla

Obiettivo

Verificare che, quando il sistema si trova nello stato GUASTO, non vengano acquisite nuove richieste.

Lo scenario controlla in particolare che la pressione del pulsante interno del piano 4, effettuata durante il guasto, non venga registrata tra le richieste attive.

Simulazione eseguita

Lo scenario porta il sistema nello stato di guasto e successivamente imposta:

```
pulsanteInterno(4) = true
```

Il sistema deve rimanere bloccato e non deve acquisire la richiesta del piano 4.

Stati osservati dalla simulazione

Dopo l'attivazione del guasto, lo stato significativo osservato è:

```
statoErrore = GUASTO
statoCabina = BLOCCATA
statoPorte = CHIUSE
direzione = NESSUNA
timer = 10
```

Dopo un ulteriore passo logico, il timer viene decrementato e la richiesta non viene acquisita:

```
statoErrore = GUASTO
statoCabina = BLOCCATA
statoPorte = CHIUSE
timer = 9
richiesteAttive(4) = false
```

Check eseguiti

Check	Esito
statoErrore = GUASTO	Passed
timer = 10	Passed
statoErrore = GUASTO dopo un passo logico	Passed
timer = 9	Passed
richiesteAttive(4) = false	Passed
statoCabina = BLOCCATA	Passed
statoPorte = CHIUSE	Passed

Coverage dello scenario

Regola	Branch coverage	Rule coverage	Update rule coverage	Forall rule coverage
r_aggiornaPersone	25.00%	25.00%	0.00%	-
r_gestisciGuasto	66.67%	62.50%	36.36%	-
r_main	75.00%	85.71%	-	-
r_acquisisciRichieste	50.00%	50.00%	0.00%	33.33%
r_gestisciErrori	50.00%	66.67%	-	-

Regole non coperte

Le seguenti regole non vengono coperte da questo scenario:

`r_gestisciOverload`

`r_gestisciAscensore`

`r_chiudiPorte`

`r_idle`

`r_scegliDirezione`

`r_muoviAscensore`

`r_serviPianoCorrente`

Questo risultato è coerente con l'obiettivo dello scenario, perché il test è focalizzato sulla gestione dello stato GUASTO.

Le regole relative a movimento, servizio dei piani, idle e overload vengono invece coperte da altri scenari.

Osservazione

Lo scenario conferma che, durante lo stato GUASTO, il modello sospende correttamente l'acquisizione di nuove richieste.

La richiesta del piano 4 non viene inserita tra le richieste attive, mentre il sistema rimane nello stato GUASTO, con cabina bloccata e porte chiuse.

9. Conservazione delle richieste durante guasto

File AVALLA: scenario_conservazione_richieste_guasto.avalla

Obiettivo

Verificare che una richiesta già acquisita prima dell'ingresso nello stato GUASTO non venga cancellata.

Lo scenario controlla in particolare che la richiesta del piano 3, registrata prima del guasto, rimanga attiva anche dopo il blocco della cabina.

Simulazione eseguita

Lo scenario acquisisce inizialmente una richiesta per il piano 3.

```
richiesteAttive(3) = true
```

Successivamente viene generato un guasto tramite:

```
eventoGuasto = true
```

Il sistema deve entrare nello stato GUASTO, bloccare la cabina e mantenere attiva la richiesta precedentemente acquisita.

Stati osservati dalla simulazione

Dopo l'acquisizione della richiesta, lo stato significativo osservato è:

```
pianoCorrente = 1
direzione = SU
statoCabina = IN_MOVIMENTO
richiesteAttive(3) = true
```

Dopo l'attivazione del guasto, la cabina viene bloccata ma la richiesta resta memorizzata:

```
statoErrore = GUASTO
statoCabina = BLOCCATA
statoPorte = CHIUSE
direzione = NESSUNA
timer = 10
richiesteAttive(3) = true
```

Check eseguiti

Check	Esito
richiesteAttive(3) = true	Passed
pianoCorrente = 1	Passed
statoErrore = GUASTO	Passed
statoCabina = BLOCCATA	Passed
statoPorte = CHIUSE	Passed

Check	Esito
direzione = NESSUNA	Passed
richiesteAttive(3) = true dopo il guasto	Passed

Coverage dello scenario

Regola	Branch coverage	Rule coverage	Update rule coverage	Forall rule coverage
r_gestisciAscensore	50.00%	70.00%	-	-
r_aggiornaPersone	25.00%	25.00%	0.00%	-
r_gestisciGuasto	50.00%	50.00%	36.36%	-
r_scegliDirezione	22.22%	26.32%	10.00%	-
r_main	75.00%	100.00%	-	-
r_acquisisciRichieste	100.00%	100.00%	100.00%	66.67%
r_muoviAscensore	33.33%	41.67%	33.33%	-
r_gestisciErrori	50.00%	66.67%	-	-

Regole non coperte

Le seguenti regole non vengono coperte da questo scenario:

`r_gestisciOverload`

`r_chiudiPorte`

`r_idle`

`r_serviPianoCorrente`

Questo risultato è coerente con l'obiettivo dello scenario, perché il test verifica la conservazione di una richiesta durante l'ingresso nello stato `GUASTO`.

Le regole relative al sovraccarico, alla chiusura delle porte, allo stato di inattività e al servizio effettivo del piano vengono invece coperte da altri scenari.

Osservazione

Lo scenario conferma che il modello conserva correttamente le richieste già acquisite prima di un guasto.

La richiesta del piano 3 rimane attiva anche dopo l'ingresso nello stato `GUASTO`, mentre la cabina viene bloccata, le porte vengono chiuse e la direzione viene annullata.

10. Ripristino dopo guasto

File AVALLA: scenario_ripristino_guasto.avalla

Obiettivo

Verificare che, dopo l'ingresso nello stato GUASTO, il sistema decrementi correttamente il timer di ripristino e torni allo stato operativo quando il timer raggiunge 0.

Lo scenario controlla che durante il guasto la cabina resti bloccata e che, al termine del timeout, lo stato di errore venga riportato a NESSUNO.

Simulazione eseguita

Lo scenario genera inizialmente un guasto tecnico.

```
eventoGuasto = true
```

Il sistema entra nello stato GUASTO e inizializza il timer a 10.

Successivamente, con il guasto non più attivo:

```
eventoGuasto = false
```

il timer viene decrementato progressivamente fino a 0.

Stati osservati dalla simulazione

Dopo l'attivazione del guasto, lo stato significativo osservato è:

```
statoErrore = GUASTO
statoCabina = BLOCCATA
statoPorte = CHIUSE
direzione = NESSUNA
timer = 10
```

Durante la fase di ripristino, il timer viene decrementato mantenendo il sistema nello stato GUASTO:

```
timer = 9
statoErrore = GUASTO
timer = 8
statoErrore = GUASTO
timer = 7
statoErrore = GUASTO
timer = 6
statoErrore = GUASTO
timer = 5
statoErrore = GUASTO
```

```
timer = 4
statoErrore = GUASTO
timer = 3
statoErrore = GUASTO
timer = 2
statoErrore = GUASTO
timer = 1
statoErrore = GUASTO
```

Quando il timer arriva a 0, il sistema risulta ancora nello stato GUASTO per quel passo logico:

```
timer = 0
statoErrore = GUASTO
```

Nel passo successivo viene completato il ripristino:

```
timer = 0
statoErrore = NESSUNO
statoCabina = FERMA
statoPorte = CHIUSE
direzione = NESSUNA
```

Check eseguiti

Check	Esito
statoErrore = GUASTO	Passed
timer = 10	Passed
timer = 9	Passed
timer = 8	Passed
timer = 7	Passed
timer = 6	Passed
timer = 5	Passed
timer = 4	Passed
timer = 3	Passed
timer = 2	Passed
timer = 1	Passed
timer = 0	Passed
statoErrore = GUASTO con timer = 0	Passed
statoErrore = NESSUNO dopo il ripristino	Passed
statoCabina = FERMA	Passed
statoPorte = CHIUSE	Passed

Check	Esito
direzione = NESSUNA	Passed

Coverage dello scenario

Regola	Branch coverage	Rule coverage	Update rule coverage	Forall rule coverage
r_gestisciAscensore	50.00%	50.00%	-	-
r_aggiornaPersone	25.00%	25.00%	0.00%	-
r_idle	-	100.00%	0.00%	-
r_gestisciGuasto	83.33%	100.00%	54.55%	-
r_main	100.00%	100.00%	-	-
r_acquisisciRichieste	50.00%	50.00%	0.00%	33.33%
r_gestisciErrori	50.00%	66.67%	-	-

Regole non coperte

Le seguenti regole non vengono coperte da questo scenario:

`r_gestisciOverload`

`r_chiudiPorte`

`r_scegliDirezione`

`r_muoviAscensore`

`r_serviPianoCorrente`

Questo risultato è coerente con l'obiettivo dello scenario, perché il test è focalizzato sul decremento del timer e sul ripristino dello stato operativo dopo un guasto.

Le regole relative al sovraccarico, alla scelta della direzione, al movimento e al servizio dei piani vengono invece coperte da altri scenari.

Osservazione

Lo scenario conferma che il modello gestisce correttamente il ripristino dopo un guasto.

Durante lo stato GUASTO, il timer viene decrementato progressivamente da 10 a 0. Quando il timer raggiunge 0, il sistema completa il ripristino nel passo logico successivo, riportando statoErrore a NESSUNO, la cabina a FERMA, le porte a CHIUSE e la direzione a NESSUNA.

Generazione automatica di scenari con ATGT

Oltre agli scenari AVALLA definiti manualmente, il modello è stato analizzato anche tramite ATGT, utilizzato per generare automaticamente ulteriori scenari di test.

L'obiettivo dell'utilizzo di ATGT è stato quello di affiancare alla validazione manuale una forma di esplorazione automatica del modello, generando sequenze di input e controlli sullo stato del sistema.

Gli scenari AVALLA manuali sono stati progettati per verificare casi specifici e significativi, come il movimento della cabina, la gestione del sovraccarico e la gestione del guasto. Gli scenari generati tramite ATGT, invece, sono stati utilizzati per esplorare ulteriori configurazioni raggiungibili del modello.

Scenari generati

testtest0.avalla

testtest2.avalla

testtest4.avalla

testtest6.avalla

testtest8.avalla

Gli scenari sono stati salvati in una cartella separata rispetto agli scenari manuali, in modo da distinguere chiaramente la validazione progettata manualmente dalla generazione automatica.

Comportamenti osservati

Gli scenari generati da ATGT hanno permesso di osservare principalmente:

- acquisizione automatica di più richieste contemporanee;
- ingresso nello stato GUASTO;
- inizializzazione del timer di ripristino;
- decremento progressivo del timer;
- mantenimento della cabina nello stato BLOCCATA durante il guasto;
- mantenimento delle porte nello stato CHIUSE durante il guasto;
- ingresso nello stato OVERLOAD;
- blocco della cabina durante il sovraccarico;
- conservazione delle richieste già acquisite.

In particolare, alcuni scenari generati attivano immediatamente l'evento di guasto, impostando:

```
eventoGuasto = true
```

In questi casi il modello porta correttamente il sistema nello stato di guasto, con cabina bloccata, porte chiuse, direzione annullata e timer inizializzato:

```
statoErrore = GUASTO
statoCabina = BLOCCATA
statoPorte = CHIUSE
direzione = NESSUNA
timer = 10
```

Nei passi successivi viene inoltre verificato il decremento progressivo del timer:

```
timer = 9
timer = 8
timer = 7
...
timer = 1
```

Altri scenari generati esplorano invece condizioni di sovraccarico, verificando che il sistema entri nello stato:

```
statoErrore = OVERLOAD
statoCabina = BLOCCATA
statoPorte = APERTE
direzione = NESSUNA
```

Differenza rispetto agli scenari manuali

Gli scenari AVALLA manuali sono stati utilizzati per verificare requisiti funzionali precisi e facilmente interpretabili.

Gli scenari generati da ATGT hanno invece un ruolo complementare: permettono di esplorare automaticamente il comportamento del modello su sequenze di input più ampie e meno guidate.

In questo senso, la validazione manuale e la generazione automatica svolgono ruoli diversi ma complementari: gli scenari manuali verificano in modo mirato i requisiti principali, mentre gli scenari generati con ATGT permettono di osservare ulteriori configurazioni del modello.

Osservazioni sui valori generati

Alcuni scenari generati da ATGT contengono valori molto elevati o negativi per le variabili *personeEntrate* e *personeUscite*.

Esempi:

```
personeEntrate := -468688956
personeUscite := -1007009643
```

Questi valori non rappresentano casi realistici dal punto di vista applicativo, ma derivano dal fatto che tali variabili sono modellate come Integer e quindi ATGT può generare valori interi arbitrari.
Nonostante ciò, gli scenari risultano utili per verificare la robustezza logica del modello rispetto a configurazioni estreme.
Questa osservazione evidenzia una possibile estensione futura del modello: limitare il dominio di *personeEntrate* e *personeUscite* a valori più realistici, ad esempio mediante domini finiti o vincoli sugli input.

Risultato complessivo

Gli scenari generati automaticamente hanno confermato la coerenza generale del modello rispetto ai comportamenti esplorati.
In particolare, il modello mantiene correttamente gli stati di sicurezza durante condizioni anomale: in stato GUASTO la cabina resta bloccata, le porte restano chiuse e il timer viene decrementato; in stato OVERLOAD la cabina resta bloccata, le porte restano aperte e la direzione viene annullata; inoltre le richieste già acquisite vengono mantenute nello stato del sistema.
ATGT è stato quindi utilizzato come supporto alla validazione manuale, aumentando la confidenza sul comportamento complessivo del modello.

Conclusione

La validazione tramite scenari AVALLA manuali e tramite scenari generati automaticamente con ATGT conferma il comportamento corretto del modello rispetto alle principali situazioni operative considerate.

Gli scenari manuali permettono di verificare in modo mirato il funzionamento ordinario dell'ascensore e la gestione delle condizioni anomale, distinguendo correttamente tra sovraccarico e guasto.

Gli scenari generati con ATGT permettono invece di esplorare automaticamente ulteriori configurazioni del modello, aumentando la confidenza nella correttezza del comportamento complessivo.

Il modello risulta quindi coerente con i requisiti funzionali e con le principali proprietà di sicurezza definite nella documentazione di progetto.